

Title: WAP to implement Caesar Cipher Algorithm.

Caesar Cipher:

Cryptography is the science of securing information by converting readable data into an unreadable format. One of the simplest and oldest encryption techniques is the Caesar Cipher, which was used by Julius Caesar to send confidential messages.

The Caesar Cipher is a substitution cipher in which each letter of the plaintext is shifted by a fixed number of positions in the alphabet. The receiver can decrypt the message by shifting the letters back by the same number of positions.

Algorithm:**Encryption**

1. Start.
2. Input the plaintext message.
3. Input the shift key.
4. For each character:
 - If it is an uppercase letter, shift within A-Z.
 - If it is a lowercase letter, shift within a-z.
 - Leave other characters unchanged.
5. Display the encrypted message.
6. Stop.

Decryption

1. Take the encrypted message.
2. Shift each character backward by the same key value.
3. Display the original message.
4. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char text[100], encrypted[100], decrypted[100];
    int key, i;
    printf("=====\n");
    printf("||  Caesar Cipher Algorithm  ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter a message: ");
    fgets(text, sizeof(text), stdin);
    // Remove newline character if present
    text[strcspn(text, "\n")] = '\0';
    printf("Enter shift key: ");
    scanf("%d", &key);
    // Encryption
    for (i = 0; text[i] != '\0'; i++)
    {
        char ch = text[i];

        if (ch >= 'A' && ch <= 'Z')
        {
            encrypted[i] = ((ch - 'A' + key) % 26) + 'A';
        }
        else if (ch >= 'a' && ch <= 'z')
        {
            encrypted[i] = ((ch - 'a' + key) % 26) + 'a';
        }
        else
        {
            encrypted[i] = ch;
        }
    }
    encrypted[i] = '\0';
    // Decryption
    for (i = 0; encrypted[i] != '\0'; i++)
    {
        char ch = encrypted[i];
        if (ch >= 'A' && ch <= 'Z')
        {
            decrypted[i] = ((ch - 'A' - key + 26) % 26) + 'A';
        }
        else if (ch >= 'a' && ch <= 'z')
        {
            decrypted[i] = ((ch - 'a' - key + 26) % 26) + 'a';
        }
        else
        {

```

```

        decrypted[i] = ch;
    }
}
decrypted[i] = '\0';

printf("\nOriginal Message : %s\n", text);
printf("Encrypted Message: %s\n", encrypted);
printf("Decrypted Message: %s\n", decrypted);

return 0;
}

```

Output:

```

=====
||      Caesar Cipher Algorithm      ||
|| Compiled by :- Jonash Chataut    ||
=====

Enter a message: CipherAlgorithm
Enter shift key: 3

Original Message : CipherAlgorithm
Encrypted Message: FlskhuDojrulwkp
Decrypted Message: CipherAlgorithm

```

Title: WAP to implement Polyalphabetic Caesar Cipher Algorithm (Vignere's).

Vignere Cipher:

The Vignere Cipher is a polyalphabetic substitution cipher that uses a keyword to encrypt plaintext. Unlike the Caesar Cipher, which uses a single shift value for all characters, the Vignere Cipher uses multiple shift values determined by the letters of the keyword. This makes it more secure than a simple Caesar Cipher because the same plaintext letter can be encrypted into different ciphertext letters depending on its position and the corresponding keyword character.

Algorithm

1. Start.
2. Input plaintext and a keyword (key).
3. Extend the key to match the plaintext length by repeating the keyword, ignoring non-alphabetic characters in the plaintext.
4. Convert each letter of the plaintext and the extended key to its corresponding number (A=0, B=1, ..., Z=25).
5. Encrypt each plaintext letter using: $C_i = (P_i + K_i) \bmod 26$, where P_i is the i-th plaintext letter's numerical value, K_i is the i-th key letter's numerical value, and C_i is the resulting ciphertext letter.
6. Convert the resulting numbers back to letters to form the ciphertext, preserving case and non-alphabetic characters.
7. To decrypt, use: $P_i = (C_i - K_i + 26) \bmod 26$, where C_i is the i-th ciphertext letter's numerical value.
8. Convert the resulting numbers back to letters to retrieve the original plaintext, preserving case and non-alphabetic characters.
9. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main()
{
    char text[100], key[100];
    char encrypted[100], decrypted[100];
    int i, j;
    printf("=====\n");
    printf("|| Vignere Cipher Algorithm ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter plaintext: ");
    fgets(text, sizeof(text), stdin);
    text[strcspn(text, "\n")] = '\0';
    printf("Enter keyword: ");
    scanf("%s", key);
```

```

int keyLen = strlen(key);
// Encryption
for (i = 0, j = 0; text[i] != '\0'; i++)
{
    if (isalpha(text[i]))
    {
        char base = isupper(text[i]) ? 'A' : 'a';
        encrypted[i] = ((toupper(text[i]) - 'A' +
            (toupper(key[j % keyLen]) - 'A')) % 26 + base;
        j++;
    }
    else
    {
        encrypted[i] = text[i];
    }
}
encrypted[i] = '\0';
// Decryption
for (i = 0, j = 0; encrypted[i] != '\0'; i++)
{
    if (isalpha(encrypted[i]))
    {
        char base = isupper(encrypted[i]) ? 'A' : 'a';
        decrypted[i] = ((toupper(encrypted[i]) - 'A' -
            (toupper(key[j % keyLen]) - 'A') + 26) % 26 + base;
        j++;
    }
    else
    {
        decrypted[i] = encrypted[i];
    }
}
decrypted[i] = '\0';
printf("\nOriginal Message : %s\n", text);
printf("Encrypted Message: %s\n", encrypted);
printf("Decrypted Message: %s\n", decrypted);
return 0;
}

```

Output:

```

=====
||   Vigenere Cipher Algorithm   ||
|| Compiled by :- Jonash Chataut ||
=====

Enter plaintext: Cryptographic
Enter keyword: BSC

Original Message : Cryptographic
Encrypted Message: Djaqlqhjcqzkd
Decrypted Message: Cryptographic

```

Title: WAP to implement Playfair Cipher

Playfair Cipher

The Playfair Cipher is a digraph substitution cipher that encrypts pairs of letters (digraphs) instead of single letters. It uses a 5×5 matrix of letters generated from a keyword. Since there are 26 letters in the English alphabet and only 25 positions in the matrix, the letters I and J are usually treated as the same letter.

The plaintext is divided into pairs of letters. If both letters in a pair are the same, a filler character (usually X) is inserted between them. If the plaintext contains an odd number of letters, an X is added at the end.

Algorithm

1. Start.
2. Input plaintext and keyword.
3. Generate a 5x5 matrix from the keyword (duplicates removed, I/J combined) and remaining alphabet (excluding J).
4. Form digraphs from plaintext: insert 'X' for identical letters or odd length, preserve non-alphabetic characters.
5. Encrypt each digraph using the matrix:
 - Same row: shift each letter right (wrap around).
 - Same column: shift each letter down (wrap around).
 - Rectangle/ Diagonal: swap each letter with the one in its row and the other's column.
6. Decrypt by reversing: shift left for same row, up for same column, same for rectangle.
7. Preserve letter case and non-alphabetic characters.
8. Stop.

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char matrix[5][5];
void generateMatrix(char key[])
{
    int used[26] = {0};
    int row = 0, col = 0;
    used['J' - 'A'] = 1;
    for (int i = 0; key[i] != '\0'; i++)
    {
        char ch = toupper(key[i]);
        if (ch == 'J')
            ch = 'I';
        if (ch >= 'A' && ch <= 'Z' && !used[ch - 'A'])
        {
            matrix[row][col++] = ch;
```

```

        used[ch - 'A'] = 1;
        if (col == 5)
        {
            row++;
            col = 0;
        }
    }
}
for (char ch = 'A'; ch <= 'Z'; ch++)
{
    if (!used[ch - 'A'])
    {
        matrix[row][col++] = ch;
        if (col == 5)
        {
            row++;
            col = 0;
        }
    }
}
}

```

```

void findPosition(char ch, int *row, int *col)
{
    if (ch == 'J')
        ch = 'I';
    for (int i = 0; i < 5; i++)
    {
        for (int j = 0; j < 5; j++)
        {
            if (matrix[i][j] == ch)
            {
                *row = i;
                *col = j;
                return;
            }
        }
    }
}
}

```

```

int main()
{
    char key[50], plaintext[100], prepared[100];
    char ciphertext[100];
    printf("=====\n");
    printf("|| Playfair Cipher ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter keyword: ");
    scanf("%s", key);
}

```

```

printf("Enter plaintext: ");
scanf("%s", plaintext);
generateMatrix(key);
int k = 0;
for (int i = 0; plaintext[i] != '\0'; i++)
{
    char ch = toupper(plaintext[i]);
    if (ch == 'J')
        ch = 'I';
    prepared[k++] = ch;
    if (plaintext[i + 1] != '\0' &&
        toupper(plaintext[i]) == toupper(plaintext[i + 1]))
    {
        prepared[k++] = 'X';
    }
}
if (k % 2 != 0)
    prepared[k++] = 'X';
prepared[k] = '\0';
int c = 0;
for (int i = 0; i < k; i += 2)
{
    int r1, c1, r2, c2;
    findPosition(prepared[i], &r1, &c1);
    findPosition(prepared[i + 1], &r2, &c2);
    if (r1 == r2)
    {
        ciphertext[c++] = matrix[r1][(c1 + 1) % 5];
        ciphertext[c++] = matrix[r2][(c2 + 1) % 5];
    }
    else if (c1 == c2)
    {
        ciphertext[c++] = matrix[(r1 + 1) % 5][c1];
        ciphertext[c++] = matrix[(r2 + 1) % 5][c2];
    }
    else
    {
        ciphertext[c++] = matrix[r1][c2];
        ciphertext[c++] = matrix[r2][c1];
    }
}
ciphertext[c] = '\0';
printf("\nPlayfair Matrix:\n");
for (int i = 0; i < 5; i++)
{
    for (int j = 0; j < 5; j++)
    {
        printf("%c ", matrix[i][j]);
    }
    printf("\n");
}

```



```

}
printf("\nPrepared Plaintext : %s\n", prepared);
printf("Encrypted Message : %s\n", ciphertext);
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  X  +  v
=====
||      Playfair Cipher      ||
|| Compiled by :- Jonash Chataut ||
=====

Enter keyword: HelloWorld
Enter plaintext: PlayfairCipher

Playfair Matrix:
H E L O W
R D A B C
F G I K M
N P Q S T
U V X Y Z

Prepared Plaintext : PLAYFAIRCIPHER
Encrypted Message  : QEBXIRFAAMNEHD

-----

```

Title: WAP to implement Hill Cipher

Hill Cipher

The Hill Cipher is a polygraphic substitution cipher that encrypts blocks of letters using linear algebra. It employs a key matrix (e.g., 2x2) to transform plaintext into ciphertext via matrix multiplication modulo 26. Each block of plaintext letters is converted to numerical values (A=0, B=1, ..., Z=25), multiplied by the key matrix, and converted back to letters. Decryption uses the inverse key matrix. The key matrix must be invertible modulo 26 (i.e., its determinant must be coprime with 26). Non-alphabetic characters remain unchanged, and padding (e.g., 'x') is added if the text length doesn't match the block size.

Algorithm

1. **Start**
2. **Input Plaintext and Key Matrix**
 - Provide plaintext message and a square invertible key matrix (e.g., 2x2 or 3x3).
3. **Convert Plaintext to Numerical Form**
 - Map letters to numbers: A=0, B=1, ..., Z=25.
4. **Divide Plaintext into Blocks**
 - Split plaintext into blocks of size n (dimension of the key matrix). □
Padding: If the last block is incomplete, add "X" to fill it.
5. **Encrypt Each Block**
 - Multiply each block (as a vector) by the key matrix: $C = K * P \bmod 26$
(C = ciphertext vector, K = key matrix, P = plaintext vector).
6. **Convert Numbers to Ciphertext**
 - Map resulting numerical vectors back to letters.
7. **Decrypt Using Inverse Key Matrix**
 - Compute the modular inverse of the key matrix ($K^{-1} \bmod 26$).
 - Multiply ciphertext vectors by K^{-1} :
 $P = K^{-1} * C \bmod 26$.
8. **Convert Numbers to Plaintext**
 - Map decrypted numerical vectors back to letters (remove padding "X" if added).
9. **Stop**

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
int main()
{
    char plaintext[100], ciphertext[100];
    int key[2][2];
    int i, j = 0;
```

```

printf("=====\n");
printf("|| Hill Cipher ||\n");
printf("|| Compiled by :- Jonash Chataut ||\n");
printf("=====\n\n");
printf("Enter plaintext: ");
scanf("%s", plaintext);
printf("Enter 2x2 Key Matrix:\n");
for (i = 0; i < 2; i++)
{
    for (int k = 0; k < 2; k++)
    {
        scanf("%d", &key[i][k]);
    }
}
int len = strlen(plaintext);
// Convert to uppercase
for (i = 0; i < len; i++)
{
    plaintext[i] = toupper(plaintext[i]);
}
// Add X if length is odd
if (len % 2 != 0)
{
    plaintext[len] = 'X';
    plaintext[len + 1] = '\0';
    len++;
}
for (i = 0; i < len; i += 2)
{
    int p1 = plaintext[i] - 'A';
    int p2 = plaintext[i + 1] - 'A';
    int c1 = (key[0][0] * p1 + key[0][1] * p2) % 26;
    int c2 = (key[1][0] * p1 + key[1][1] * p2) % 26;
    ciphertext[j++] = c1 + 'A';
    ciphertext[j++] = c2 + 'A';
}
ciphertext[j] = '\0';
printf("\nPlaintext : %s\n", plaintext);
printf("Ciphertext : %s\n", ciphertext);
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  ×  +  ▾

=====
|| Hill Cipher ||
|| Compiled by :- Jonash Chataut ||
=====

Enter plaintext: WorldCup
Enter 2x2 Key Matrix:
6 7
7 6

Plaintext : WORLD CUP
Ciphertext : WEXDGHRW

```

Title: WAP to implement Rail fence Cipher

Rail Fence Cipher:

The Rail Fence Cipher is a type of transposition cipher in which the plaintext is written in a zigzag pattern across multiple rows (rails) and then read row by row to produce the ciphertext. Unlike substitution ciphers, the Rail Fence Cipher does not replace characters; instead, it changes their positions according to a specific pattern.

The number of rails acts as the key for encryption and decryption. This cipher is simple to implement and demonstrates the concept of transposition-based cryptography.

Algorithm:

The Rail Fence Cipher operates as follows:

1. Encryption:

- The plaintext is written in a zigzag pattern across a fixed number of rails (rows), determined by the key.
- The pattern alternates between moving down and up the rails, repeating every $2 * (\text{rails} - 1)$ positions.
- The ciphertext is formed by reading the characters row by row from the resulting grid.

2. Decryption:

- The ciphertext is used to reconstruct the zigzag pattern by determining the positions of characters in the grid based on the number of rails.
- The plaintext is recovered by reading the characters column by column, following the same zigzag path used during encryption.

3. Key Considerations:

- The number of rails must be at least 2 for the cipher to produce a meaningful rearrangement.
- The cipher preserves all characters, including non-alphabetic ones, ensuring the text structure remains intact.
- The security of the cipher depends on the number of rails and the length of the text, though it is relatively weak compared to modern encryption methods due to its predictable pattern.

Source Code:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char text[100], cipher[100], decrypt[100];
    int rails, len, i, j;
    printf("=====\n");
    printf("|| Rail Fence Cipher ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter plaintext: ");
    fgets(text, sizeof(text), stdin);
```

```

text[strcspn(text, "\n")] = '\0';
len = strlen(text);
printf("Enter number of rails: ");
scanf("%d", &rails);
char rail[rails][100];
// Initialize matrix
for (i = 0; i < rails; i++)
    for (j = 0; j < len; j++)
        rail[i][j] = '\n';
// Encryption
int row = 0, dir = 1;
for (i = 0; i < len; i++)
{
    rail[row][i] = text[i];
    if (row == 0)
        dir = 1;
    else if (row == rails - 1)
        dir = -1;
    row += dir;
}
int k = 0;
for (i = 0; i < rails; i++)
{
    for (j = 0; j < len; j++)
    {
        if (rail[i][j] != '\n')
            cipher[k++] = rail[i][j];
    }
}
cipher[k] = '\0';
// Decryption
for (i = 0; i < rails; i++)
    for (j = 0; j < len; j++)
        rail[i][j] = '\n';
row = 0;
dir = 1;
for (i = 0; i < len; i++)
{
    rail[row][i] = '*';
    if (row == 0)
        dir = 1;
    else if (row == rails - 1)
        dir = -1;
    row += dir;
}
k = 0;
for (i = 0; i < rails; i++)
{
    for (j = 0; j < len; j++)
    {
        if (rail[i][j] == '*' && k < len)

```

```

        rail[i][j] = cipher[k++];
    }
}
row = 0;
dir = 1;
for (i = 0; i < len; i++)
{
    decrypt[i] = rail[row][i];
    if (row == 0)
        dir = 1;
    else if (row == rails - 1)
        dir = -1;
    row += dir;
}
decrypt[len] = '\0';
printf("\nOriginal Message : %s\n", text);
printf("Encrypted Message: %s\n", cipher);
printf("Decrypted Message: %s\n", decrypt);
return 0;
}

```

Output:

The screenshot shows a terminal window with a dark theme. The title bar indicates the file path is C:\csit\5_fifth sem\Jonash Cry. The program output is as follows:

```

=====
||      Rail Fence Cipher      ||
|| Compiled by :- Jonash Chataut ||
=====

Enter plaintext: Credentials
Enter number of rails: 3

Original Message : Credentials
Encrypted Message: Ceardnilets
Decrypted Message: Credentials

-----
Process exited after 23.55 seconds with return value 0
Press any key to continue . . . |

```

Title: WAP to implement Euclidean Algorithm.

Euclidean Algorithm

The Euclidean Algorithm is an efficient method used to find the Greatest Common Divisor (GCD) of two integers. The GCD of two numbers is the largest positive integer that divides both numbers without leaving a remainder. The algorithm is based on the principle that the GCD of two numbers does not change if the larger number is replaced by its remainder when divided by the smaller number.

Mathematically:

$$\text{GCD}(a,b)=\text{GCD}(b,a\text{mod}b)$$

The process continues until the remainder becomes zero. The last non-zero divisor is the GCD.

Algorithm

1. Start.
2. Input two integers a and b.
3. Repeat the following steps while b is not equal to 0:
 - a. Compute the remainder:
 $r = a \text{ mod } b$
 - b. Assign:
 $a = b$
 - c. Assign:
 $b = r$
4. When b becomes 0, a contains the Greatest Common Divisor (GCD).
5. Display the value of a as the GCD.
6. Stop.

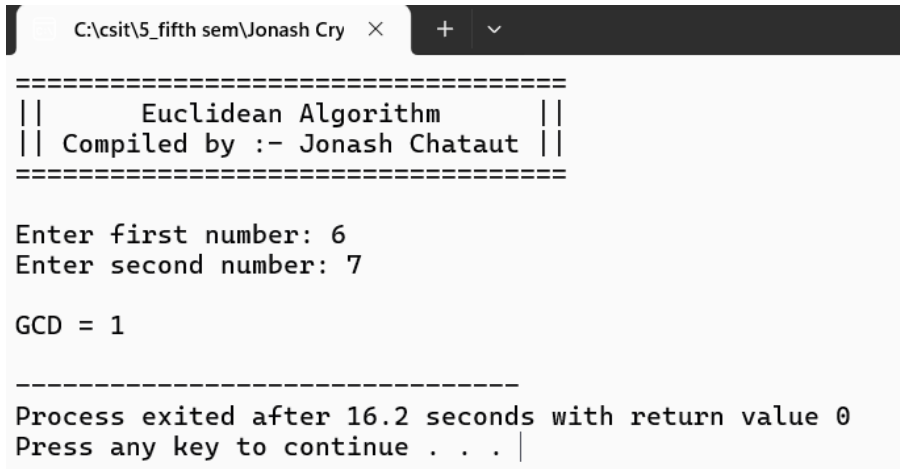
Source Code:

```
#include <stdio.h>
int main()
{
    int a, b, temp;
    printf("=====\n");
    printf("|| Euclidean Algorithm ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter first number: ");
    scanf("%d", &a);
    printf("Enter second number: ");
    scanf("%d", &b);
    while (b != 0)
    {
        temp = b;
        b = a % b;
        a = temp;
    }
}
```

```
printf("\nGCD = %d\n", a);

return 0;
}
```

Output:



```
C:\csit\5_fifth sem\Jonash Cry  ×  +  v

=====
||      Euclidean Algorithm      ||
|| Compiled by :- Jonash Chataut ||
=====

Enter first number: 6
Enter second number: 7

GCD = 1

-----
Process exited after 16.2 seconds with return value 0
Press any key to continue . . . |
```


Title: WAP to implement Extended Euclidean Algorithm

Extended Euclidean Algorithm:

The Extended Euclidean Algorithm is an extension of the Euclidean Algorithm that not only finds the Greatest Common Divisor (GCD) of two integers a and b , but also finds two integers x and y such that: $ax+by=\text{gcd}(a,b)$.

These values x and y are known as Bézout coefficients. The Extended Euclidean Algorithm is widely used in cryptography, particularly in algorithms like RSA for finding the modular multiplicative inverse.

Algorithm

1. Start.
2. Input two integers a and b .
3. If $b = 0$:
 - a. Set $\text{gcd} = a$.
 - b. Set $x = 1$ and $y = 0$.
 - c. Return gcd , x , and y .
4. Otherwise:
 - a. Recursively call `ExtendedEuclidean(b, a mod b)`.
 - b. Let gcd , x_1 , y_1 be the returned values.
 - c. Compute:
 $x = y_1$
 $y = x_1 - (a / b) * y_1$
5. Return gcd , x , and y .
6. Display gcd , x , and y .
7. Stop.

Source Code:

```
#include <stdio.h>
int extendedGCD(int a, int b, int *x, int *y)
{
    if (b == 0)
    {
        *x = 1;
        *y = 0;
        return a;
    }
    int x1, y1;
    int gcd = extendedGCD(b, a % b, &x1, &y1);
    *x = y1;
    *y = x1 - (a / b) * y1;
    return gcd;
}
int main()
{
    int a, b, x, y, gcd;
```

```

printf("=====\n");
printf("|| Extended Euclidean Algorithm ||\n");
printf("|| Compiled by :- Jonash Chataut ||\n");
printf("=====\n\n");
printf("Enter first number: ");
scanf("%d", &a);
printf("Enter second number: ");
scanf("%d", &b);
gcd = extendedGCD(a, b, &x, &y);
printf("\nGCD = %d\n", gcd);
printf("x = %d\n", x);
printf("y = %d\n", y);
printf("\nVerification:\n");
printf("%d(%d) + %d(%d) = %d\n", a, x, b, y, gcd);
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  X  +  v
=====
|| Extended Euclidean Algorithm ||
|| Compiled by :- Jonash Chataut ||
=====

Enter first number: 89
Enter second number: 9

GCD = 1
x = -1
y = 10

Verification:
89(-1) + 9(10) = 1
-----

```

Title: WAP to implement Miller-Rabin Primality Test

Miller-Rabin Primality Test

The Miller-Rabin Primality Test is a probabilistic algorithm used to determine whether a given number is prime or composite. Unlike simple trial division methods, Miller-Rabin is much faster for large numbers and is widely used in cryptographic applications such as RSA.

The algorithm is based on properties of modular arithmetic and repeatedly tests whether a number behaves like a prime number. If a number fails any test, it is definitely composite. If it passes all tests, it is considered probably prime.

Algorithm

The Miller-Rabin Primality Test operates as follows:

1. Input:

- Take a number n to test for primality and k , the number of iterations.
- Handle edge cases: if $n = 2$, it is prime; if $n < 2$ or n is even, it is composite.

2. Factorization:

- Express $n - 1$ as $d * 2^s$, where d is odd and s is the number of times 2 divides $n - 1$.

3. Testing Loop:

- For k iterations:
 - Select a random base a in the range $[2, n-2]$.
 - Compute $x = a^d \bmod n$ using modular exponentiation.
 - If $x = 1$ or $x = n - 1$, continue to the next iteration.
 - For r from 0 to $s-1$, compute $x = x^2 \bmod n$. If $x = n - 1$, continue to the next iteration.
 - If none of these conditions are met, n is composite.

4. Output:

- If all iterations pass, n is likely prime.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
long long power(long long x, long long y, long long p)
{
    long long result = 1;
    x = x % p;
    while (y > 0)
    {
        if (y & 1)
            result = (result * x) % p;
        y = y >> 1;
        x = (x * x) % p;
    }
}
```

```

    }
    return result;
}
int millerTest(long long d, long long n)
{
    long long a = 2 + rand() % (n - 4);
    long long x = power(a, d, n);
    if (x == 1 || x == n - 1)
        return 1;
    while (d != n - 1)
    {
        x = (x * x) % n;
        d *= 2;

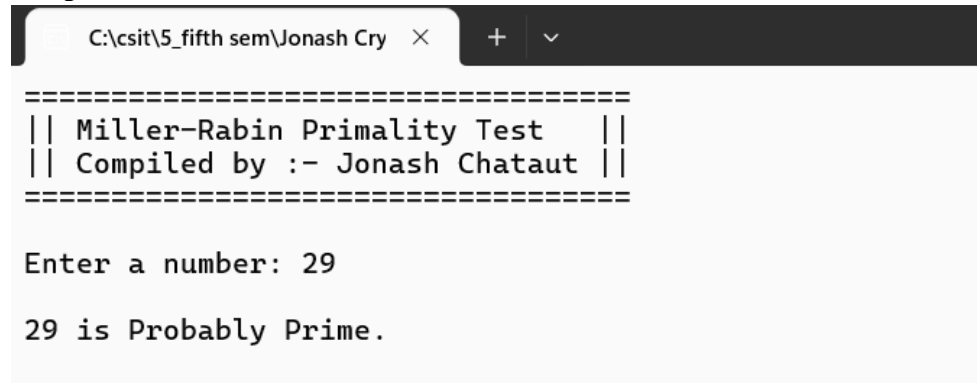
        if (x == 1)
            return 0;

        if (x == n - 1)
            return 1;
    }
    return 0;
}
int isPrime(long long n, int k)
{
    if (n <= 1 || n == 4)
        return 0;
    if (n <= 3)
        return 1;
    long long d = n - 1;
    while (d % 2 == 0)
        d /= 2;
    for (int i = 0; i < k; i++)
    {
        if (!millerTest(d, n))
            return 0;
    }
    return 1;
}
int main()
{
    long long n;
    int k = 5;
    printf("=====\n");
    printf("|| Miller-Rabin Primality Test  ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter a number: ");
    scanf("%lld", &n);
    if (isPrime(n, k))
        printf("\n%lld is Probably Prime.\n", n);
    else

```

```
        printf("\n%lld is Composite.\n", n);  
    return 0;  
}
```

Output:



```
C:\csit\5_fifth sem\Jonash Cry  ×  +  ▾  
  
=====  
|| Miller-Rabin Primality Test  ||  
|| Compiled by :- Jonash Chataut ||  
=====  
  
Enter a number: 29  
  
29 is Probably Prime.
```

Title: WAP to implement Euler's Theorem

Euler's Theorem

Euler's Theorem is a fundamental result in number theory, named after Leonhard Euler, which states that if two integers a and n are coprime (i.e., their greatest common divisor is 1), then a raised to the power of Euler's totient function of n , denoted $\phi(n)$, is congruent to 1 modulo n : $a^{\phi(n)} \equiv 1 \pmod{n}$. The totient function $\phi(n)$ counts the number of integers from 1 to n that are coprime with n . For a prime number p , $\phi(p) = p - 1$, and for a product of distinct primes, $\phi(n)$ is computed as n times the product of $(1 - 1/p_i)$ for each prime factor p_i of n . This theorem is a generalization of Fermat's Little Theorem and is crucial in cryptographic algorithms like RSA, where it underpins modular exponentiation and the computation of modular inverses.

Algorithm

Euler's Theorem is demonstrated as follows:

- 1. Input:** Take two integers: a (the base) and n (the modulus, positive).
- 2. Validation:**
 - Check if $n > 0$; if not, the modulus is invalid.
 - Check if $a > 0$; if not, the base is invalid.
 - Compute $\text{GCD}(a, n)$ using the Euclidean Algorithm; if $\text{GCD} \neq 1$, a and n are not coprime, and the theorem does not apply.
- 3. Compute Euler's Totient Function ($\phi(n)$):**
 - Initialize $\text{result} = n$.
 - For each prime factor p of n (found by checking divisors up to $\sqrt{n}$):
 - If n is divisible by p , divide n by p repeatedly and update result as $\text{result} - \text{result}/p$.
 - If $n > 1$ after the loop, account for the remaining prime factor.
- 4. Modular Exponentiation:**
 - Compute $a^{\phi(n)} \bmod n$ using an efficient modular exponentiation algorithm.
- 5. Output:**
 - If a and n are coprime, the result should be 1, verifying Euler's Theorem.
 - Handle error cases (invalid modulus, base, or non-coprime inputs).
- 6. Key Considerations:**
 - The algorithm assumes efficient computation of $\phi(n)$, with a time complexity of $O(\sqrt{n})$ for factorization.
 - Modular exponentiation is performed in $O(\log \phi(n) * \log n)$ using the square-and-multiply method.
 - The theorem is only valid when a and n are coprime, making GCD computation critical.

Source Code:

```
#include <stdio.h>
int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
int phi(int n)
{
    int result = n;
    for (int p = 2; p * p <= n; p++)
    {
        if (n % p == 0)
        {
            while (n % p == 0)
                n /= p;

            result -= result / p;
        }
    }
    if (n > 1)
        result -= result / n;

    return result;
}
long long powerMod(long long a, long long b, long long mod)
{
    long long result = 1;
    while (b > 0)
    {
        if (b % 2 == 1)
            result = (result * a) % mod;
        a = (a * a) % mod;
        b /= 2;
    }
    return result;
}
int main()
{
    int a, n;
    printf("=====\n");
    printf("|| Euler's Theorem ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter value of a: ");
    scanf("%d", &a);
```

```

printf("Enter value of n: ");
scanf("%d", &n);
if (gcd(a, n) != 1)
{
    printf("\nEuler's Theorem is not applicable.\n");
    printf("gcd(%d, %d) != 1\n", a, n);
    return 0;
}
int totient = phi(n);
long long result = powerMod(a, totient, n);
printf("\nEuler Totient  $\phi$ (%d) = %d\n", n, totient);
printf("%d^%d mod %d = %lld\n", a, totient, n, result);
if (result == 1)
    printf("\nEuler's Theorem Verified.\n");
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  ×  +  v
=====
||      Euler's Theorem      ||
|| Compiled by :- Jonash Chataut ||
=====

Enter value of a: 96
Enter value of n: 5

Euler Totient  $\phi$ (5) = 4
96^4 mod 5 = 1

Euler's Theorem Verified.
-----

```


Title: WAP to find the primitive roots of a number

Primitive Roots

In number theory, a primitive root modulo a prime number p is an integer a (where $1 \leq a < p$) such that the powers $a^1, a^2, \dots, a^{(p-1)}$ modulo p generate all integers from 1 to $p-1$ in some order. This implies that the order of a modulo p , defined as the smallest positive integer k such that $a^k \equiv 1 \pmod{p}$, must equal $p-1$. Primitive roots exist only for certain moduli, notably prime numbers, and are significant in cryptographic systems like Diffie-Hellman key exchange and discrete logarithm problems. For a prime p , the number of primitive roots is given by Euler's totient function $\phi(p-1)$, and they are found by testing whether $a^i \bmod p \neq 1$ for all i from 1 to $p-2$ and verifying that $a^{(p-1)} \bmod p = 1$, per Fermat's Little Theorem.

Algorithm

The process to find primitive roots of a prime number p is as follows:

1. Input: Take a positive integer p and verify it is prime.
2. Validation:
 - If $p \leq 1$, it is invalid.
 - If p is not prime, primitive roots do not exist (except for $p = 2$, where 1 is the primitive root).
 - If $p = 2$, output 1 as the only primitive root.
3. Primitive Root Check:
 - For each integer a from 1 to $p-1$:
 - Compute $a^i \bmod p$ for i from 1 to $p-2$ using modular exponentiation.
 - If any $a^i \bmod p = 1$, a is not a primitive root.
 - If a passes the check, verify $a^{(p-1)} \bmod p = 1$.
 - If both conditions hold, a is a primitive root.
4. Output:
 - List all primitive roots found or indicate if none exist (unlikely for a prime).
5. Key Considerations:
 - The algorithm uses modular exponentiation for efficiency, with a time complexity of $O(p * \log p * \log(p-1))$ for testing all candidates.
 - Primality testing is performed using trial division up to $\sqrt{p}$, with complexity $O(\sqrt{p})$.
 - The number of primitive roots depends on $\phi(p-1)$, and for large p , testing can be optimized by checking only prime factors of $p-1$.

Source Code:

```
#include <stdio.h>
int power(int x, unsigned int y, int p)
{
    int result = 1;
    x = x % p;
    while (y > 0)
    {
```

```

        if (y & 1)
            result = (result * x) % p;
        y = y >> 1;
        x = (x * x) % p;
    }
    return result;
}
int phi(int n)
{
    int result = n;

    for (int p = 2; p * p <= n; p++)
    {
        if (n % p == 0)
        {
            while (n % p == 0)
                n /= p;

            result -= result / p;
        }
    }
    if (n > 1)
        result -= result / n;

    return result;
}
int main()
{
    int n;
    printf("=====\n");
    printf("|| Primitive Root Finder ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter a number: ");
    scanf("%d", &n);
    int phi_n = phi(n);
    int factors[100], count = 0;
    int temp = phi_n;
    for (int i = 2; i * i <= temp; i++)
    {
        if (temp % i == 0)
        {
            factors[count++] = i;
            while (temp % i == 0)
                temp /= i;
        }
    }
    if (temp > 1)
        factors[count++] = temp;
    printf("\nPrimitive Roots of %d are:\n", n);
    for (int g = 2; g < n; g++)

```

```

{
    int flag = 1;
    for (int i = 0; i < count; i++)
    {
        if (power(g, phi_n / factors[i], n) == 1)
        {
            flag = 0;
            break;
        }
    }
    if (flag)
        printf("%d ", g);
}
printf("\n");
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  ×  +  ▾

=====
||      Primitive Root Finder      ||
|| Compiled by :- Jonash Chataut  ||
=====

Enter a number: 13

Primitive Roots of 13 are:
2 6 7 11

```

Title: WAP to compute the discrete logarithm of a number

Discrete Logarithm

The Discrete Logarithm Problem (DLP) is the inverse operation of modular exponentiation. Given a base g , a modulus p , and a value h , the objective is to find an integer x such that:

$$g^x \equiv h \pmod{p}$$

The value x is called the discrete logarithm of h to the base g modulo p . Unlike ordinary logarithms, discrete logarithms are computed in modular arithmetic and are generally difficult to solve for large numbers. This computational difficulty forms the basis of many cryptographic systems, including Diffie-Hellman Key Exchange, ElGamal Encryption, and other public-key cryptographic algorithms. For small values, the discrete logarithm can be found by repeatedly computing powers of g modulo p until the result equals h . The exponent at which this occurs is the discrete logarithm.

Algorithm

The computation of the discrete logarithm operates as follows:

1. Input:

- Take three integers: a (the base, a primitive root), b (the target number), and p (a prime modulus).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Ensure a and b are positive and $b \not\equiv 0 \pmod{p}$.
- Confirm a is a primitive root of p by checking that $a^i \pmod{p} \neq 1$ for i from 1 to $p-2$ and $a^{(p-1)} \pmod{p} = 1$.

3. Computation:

- Use a brute-force approach: for i from 0 to $p-1$, compute $a^i \pmod{p}$ using modular exponentiation.
- If $a^i \pmod{p} = b \pmod{p}$, then i is the discrete logarithm.

4. Output:

- Return the discrete logarithm i if found, or indicate invalid inputs (nonprime p , non-primitive root a , or no solution).

5. Key Considerations:

- The brute-force approach has a time complexity of $O(p * \log p)$ due to modular exponentiation.
- For large p , more efficient algorithms like the Baby-step Giant-step or Pollard's Rho are preferred, though not implemented here.
- The algorithm assumes a is a primitive root, ensuring a unique solution exists for each b in $\{1, 2, \dots, p-1\}$.

Source Code:

```
#include <stdio.h>
long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;
    while (exp > 0)
```

```

{
    if (exp % 2 == 1)
        result = (result * base) % mod;
    base = (base * base) % mod;
    exp /= 2;
}
return result;
}
int main()
{
    int g, h, p;
    int found = 0;
    printf("=====\n");
    printf("|| Discrete Logarithm ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter base (g): ");
    scanf("%d", &g);
    printf("Enter value (h): ");
    scanf("%d", &h);
    printf("Enter modulus (p): ");
    scanf("%d", &p);
    for (int x = 0; x < p; x++)
    {
        if (powerMod(g, x, p) == h)
        {
            printf("\nDiscrete Logarithm = %d\n", x);
            found = 1;
            break;
        }
    }
    if (!found)
        printf("\nDiscrete logarithm does not exist.\n");
    return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  ×  +  ▾

=====
|| Discrete Logarithm ||
|| Compiled by :- Jonash Chataut ||
=====

Enter base (g): 2
Enter value (h): 8
Enter modulus (p): 11

Discrete Logarithm = 3

```

Title: WAP to implement Diffie-Hellman Key Exchange.

Diffie-Hellman Key Exchange

Diffie-Hellman Key Exchange is a public-key cryptographic protocol that allows two parties to establish a shared secret key over an insecure communication channel. It was proposed by Whitfield Diffie and Martin Hellman in 1976.

The algorithm uses two publicly known numbers: a prime number p and a primitive root g of p . Each user selects a private key and computes a corresponding public key. The public keys are exchanged, and each user computes the shared secret key using their private key and the other user's public key.

The security of the Diffie-Hellman Key Exchange is based on the difficulty of solving the Discrete Logarithm Problem.

Algorithm

The Diffie-Hellman Key Exchange operates as follows:

1. Input:

- Select a prime number p and a primitive root g modulo p .
- Alice chooses a private key a , and Bob chooses a private key b (both positive integers).

2. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Confirm g is a primitive root of p by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure private keys a and b are positive.

3. Public Key Computation:

- Alice computes her public key: $A = g^a \bmod p$.
- Bob computes his public key: $B = g^b \bmod p$.

4. Key Exchange:

- Alice and Bob exchange their public keys A and B .

5. Shared Secret Computation:

- Alice computes the shared secret: $S = B^a \bmod p$.
- Bob computes the shared secret: $S = A^b \bmod p$.
- Since $B^a \bmod p = (g^b)^a \bmod p = g^{(a*b)} \bmod p = (g^a)^b \bmod p = A^b \bmod p$, both compute the same secret.

6. Output:

- Output the shared secret if valid, or indicate errors (non-prime p , invalid g , or negative private keys).

7. Key Considerations:

- The algorithm's time complexity is $O(\log p * \log(a \text{ or } b))$ per modular exponentiation.
- Security depends on the difficulty of the discrete logarithm problem for large p .
- The protocol assumes a secure channel for public key exchange to prevent man-in-the-middle attacks.

Source Code:

```
#include <stdio.h>
long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main()
{
    long long p, g;
    long long a, b;
    long long A, B;
    long long keyA, keyB;
    printf("=====\n");
    printf("|| Diffie-Hellman Key Exchange ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter prime number (p): ");
    scanf("%lld", &p);
    printf("Enter primitive root (g): ");
    scanf("%lld", &g);
    printf("Enter private key of User A: ");
    scanf("%lld", &a);
    printf("Enter private key of User B: ");
    scanf("%lld", &b);
    A = powerMod(g, a, p);
    B = powerMod(g, b, p);
    keyA = powerMod(B, a, p);
    keyB = powerMod(A, b, p);
    printf("\nPublic Key of User A = %lld\n", A);
    printf("Public Key of User B = %lld\n", B);
    printf("\nShared Secret Key at User A = %lld\n", keyA);
    printf("Shared Secret Key at User B = %lld\n", keyB);
    if (keyA == keyB)
        printf("\nKey Exchange Successful.\n");
    else
        printf("\nKey Exchange Failed.\n");
    return 0;
}
```

Output:

```
C:\csit\5_fifth sem\Jonash Cry × + v
=====
|| Diffie-Hellman Key Exchange ||
|| Compiled by :- Jonash Chataut ||
=====

Enter prime number (p): 23
Enter primitive root (g): 5
Enter private key of User A: 7
Enter private key of User B: 67

Public Key of User A = 17
Public Key of User B = 5

Shared Secret Key at User A = 17
Shared Secret Key at User B = 17

Key Exchange Successful.
```


Title: WAP to illustrate Man in Middle Attack

Man-in-the-Middle Attack on Diffie-Hellman Key Exchange

A Man-in-the-Middle (MITM) Attack is a type of cyberattack in which an attacker secretly intercepts and possibly alters the communication between two parties who believe they are communicating directly with each other. The attacker positions themselves between the sender and receiver, allowing them to eavesdrop, modify, or inject messages without either party being aware.

In the context of the Diffie-Hellman Key Exchange, a MITM attack occurs when an attacker intercepts the public keys exchanged between two users and replaces them with their own public keys. As a result, both users establish separate secret keys with the attacker instead of with each other. The attacker can then decrypt, read, modify, and re-encrypt messages before forwarding them.

This attack highlights the importance of authentication mechanisms such as digital certificates and public key infrastructure (PKI), which help verify the identity of communicating parties.

Algorithm

The Man-in-the-Middle attack on the Diffie-Hellman Key Exchange operates as follows:

1. Setup:

- Alice and Bob agree on public parameters: a prime number p and a primitive root g modulo p .
- Alice selects a private key a , and Bob selects a private key b .
- Eve, the attacker, selects two private keys, e_1 (for communication with Bob) and e_2 (for communication with Alice).

2. Public Key Exchange and Interception:

- Alice computes her public key $A = g^a \bmod p$ and sends it to Bob, but Eve intercepts A and sends her own public key $E_2 = g^{e_2} \bmod p$ to Alice.
- Bob computes his public key $B = g^b \bmod p$ and sends it to Alice, but Eve intercepts B and sends her own public key $E_1 = g^{e_1} \bmod p$ to Bob.

3. Shared Secret Computation:

- Alice computes a shared secret with Eve: $S_1 = E_2^a \bmod p = g^{(e_2 * a)} \bmod p$.
- Bob computes a shared secret with Eve: $S_2 = E_1^b \bmod p = g^{(e_1 * b)} \bmod p$.
- Eve computes her shared secret with Alice: $S_{\text{eve_alice}} = A^{e_2} \bmod p = g^{(a * e_2)} \bmod p$ (matches S_1).
- Eve computes her shared secret with Bob: $S_{\text{eve_bob}} = B^{e_1} \bmod p = g^{(b * e_1)} \bmod p$ (matches S_2).

4. Communication:

- Alice encrypts messages using S_1 , which Eve can decrypt using $S_{\text{eve_alice}}$, modify if desired, and re-encrypt using $S_{\text{eve_bob}}$ to send to Bob.
- Similarly, Bob encrypts messages using S_2 , which Eve can decrypt and manipulate.

5. Output:

- Display the public keys, private keys, and shared secrets to demonstrate that Eve has successfully established separate shared secrets with Alice and Bob.

6. Key Considerations:

- The attack requires Eve to actively intercept and replace public keys during the exchange, with a time complexity of $O(\log p)$ per modular exponentiation.
- The basic Diffie-Hellman protocol is vulnerable because it lacks authentication; solutions like digital signatures, certificates, or password-authenticated key agreements (e.g., Station-to-Station protocol) can mitigate this.
- The attack remains undetected unless Alice and Bob compare their shared secrets, which requires a separate secure channel.

Source Code:

```
#include <stdio.h>
long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

int main()
{
    long long p, g;
    long long a, b, m;
    long long A, B, M;
    long long KA, KB;
    long long KMA, KMB;
    printf("=====\n");
    printf("|| Man in the Middle Attack ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter prime number (p): ");
    scanf("%lld", &p);
    printf("Enter primitive root (g): ");
    scanf("%lld", &g);
    printf("Enter private key of User A: ");
    scanf("%lld", &a);
    printf("Enter private key of User B: ");
    scanf("%lld", &b);
```

```

printf("Enter private key of Attacker M: ");
scanf("%lld", &m);
A = powerMod(g, a, p);
B = powerMod(g, b, p);
M = powerMod(g, m, p);
KA = powerMod(M, a, p);
KB = powerMod(M, b, p);
KMA = powerMod(A, m, p);
KMB = powerMod(B, m, p);
printf("\nPublic Key of User A = %lld\n", A);
printf("Public Key of User B = %lld\n", B);
printf("Public Key of Attacker = %lld\n", M);
printf("\nKey established by User A = %lld\n", KA);
printf("Key established by User B = %lld\n", KB);
printf("\nAttacker's Key with A = %lld\n", KMA);
printf("Attacker's Key with B = %lld\n", KMB);
printf("\nCommunication can be intercepted and modified by the attacker.\n");
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry × + v
=====
||   Man in the Middle Attack   ||
|| Compiled by :- Jonash Chataut ||
=====

Enter prime number (p): 23
Enter primitive root (g): 5
Enter private key of User A: 7
Enter private key of User B: 67
Enter private key of Attacker M: 49

Public Key of User A = 17
Public Key of User B = 5
Public Key of Attacker = 20

Key established by User A = 21
Key established by User B = 20

Attacker's Key with A = 21
Attacker's Key with B = 20

Communication can be intercepted and modified by the attacker.

=====

```

Title: WAP to implement RSA Algorithm

RSA Algorithm

RSA (Rivest-Shamir-Adleman) is one of the most widely used public-key cryptographic algorithms. It was developed by Ron Rivest, Adi Shamir, and Leonard Adleman in 1977.

RSA uses two keys:

- Public Key for encryption.
- Private Key for decryption.

The algorithm is based on the mathematical difficulty of factoring large prime numbers. Two prime numbers are selected to generate the public and private keys. A message encrypted with the public key can only be decrypted using the corresponding private key.

RSA is widely used in secure communication, digital signatures, SSL/TLS certificates, and e-commerce applications.

Algorithm

The RSA algorithm operates as follows:

1. Key Generation:

- Select two large prime numbers p and q .
- Compute the modulus $n = p * q$.
- Compute Euler's totient function $\phi(n) = (p-1)(q-1)$.
- Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
- Compute the private exponent d using the modular inverse: $d * e \equiv 1 \pmod{\phi(n)}$.

2. Encryption:

- Take a message m (where $0 \leq m < n$).
- Compute the ciphertext $c = m^e \bmod n$ using modular exponentiation.

3. Decryption:

- Compute the original message $m = c^d \bmod n$ using modular exponentiation.

4. Validation:

- Verify p and q are prime using a primality test (e.g., trial division up to $\sqrt{n}$).
- Ensure e is coprime with $\phi(n)$ using the Euclidean algorithm.
- Check that m is in the range $[0, n)$.
- Confirm d exists by computing the modular inverse of e modulo $\phi(n)$.

5. Output:

- Display the public key (e, n) , private key (d, n) , ciphertext c , and decrypted message m .
- Verify decryption success if the decrypted message matches the original.

Source Code:

```
#include <stdio.h>
long long gcd(long long a, long long b)
{
    while (b != 0)
    {
        long long temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
long long modInverse(long long e, long long phi)
{
    long long d;
    for (d = 1; d < phi; d++)
    {
        if ((d * e) % phi == 1)
            return d;
    }
    return -1;
}
long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}
int main()
{
    long long p, q, n, phi;
    long long e, d;
    long long message;
    long long ciphertext, decrypted;
    printf("=====\n");
    printf("|| RSA Algorithm ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter prime number p: ");
    scanf("%lld", &p);
    printf("Enter prime number q: ");
    scanf("%lld", &q);
    n = p * q;
    phi = (p - 1) * (q - 1);
```

```

printf("Enter public exponent e: ");
scanf("%lld", &e);
if (gcd(e, phi) != 1)
{
    printf("\nInvalid value of e.\n");
    return 0;
}
d = modInverse(e, phi);
printf("\nPublic Key = (%lld, %lld)\n", e, n);
printf("Private Key = (%lld, %lld)\n", d, n);
printf("\nEnter message: ");
scanf("%lld", &message);
ciphertext = powerMod(message, e, n);
decrypted = powerMod(ciphertext, d, n);
printf("\nEncrypted Message = %lld\n", ciphertext);
printf("Decrypted Message = %lld\n", decrypted);
return 0;
}

```

Output:

```

C:\csit\5_fifth sem\Jonash Cry  X  +  v
=====
||      RSA Algorithm      ||
|| Compiled by :- Jonash Chataut ||
=====

Enter prime number p: 11
Enter prime number q: 13
Enter public exponent e: 67

Public Key  = (67, 143)
Private Key = (43, 143)

Enter message: 49

Encrypted Message = 36
Decrypted Message = 49

-----

```

Title: WAP to implement ElGamal Cryptographic System Algorithm

ElGamal Cryptographic System

The ElGamal Cryptographic System is a public-key cryptosystem based on the difficulty of the discrete logarithm problem. It was proposed by Taher Elgamal in 1985. It is widely used for secure communication and digital signatures. The system consists of three phases: key generation, encryption, and decryption. It uses modular exponentiation and random numbers to provide strong security. In ElGamal, a large prime number p and a primitive root g are chosen. The private key is a random number x , and the public key is computed as:

$$y = g^x \bmod p$$

To encrypt a message m , a random number k is selected, and ciphertext is generated as:

$$\begin{aligned} c1 &= g^k \bmod p \\ c2 &= m \times y^k \bmod p \end{aligned}$$

To decrypt the message:

$$m = c2 \times (c1^x)^{(-1)} \bmod p$$

Algorithm

The ElGamal cryptographic system operates as follows:

1. Key Generation:

- Select a large prime p and a primitive root g modulo p .
- Choose a private key a ($1 \leq a \leq p-2$).
- Compute the public key $h = g^a \bmod p$ using modular exponentiation.

2. Encryption:

- Take a message m ($0 \leq m < p$) and a random integer k ($1 \leq k \leq p-2$).
- Compute $c1 = g^k \bmod p$. ○ Compute $c2 = m * h^k \bmod p$.
- Output the ciphertext pair $(c1, c2)$.

3. Decryption:

- Compute $s = c1^a \bmod p$.
- Compute the modular inverse $s^{(-1)} \bmod p$ using the Extended Euclidean Algorithm.
- Recover the message $m = c2 * s^{(-1)} \bmod p$.

4. Validation:

- Verify p is prime using a primality test (e.g., trial division up to $\sqrt{p}$).
- Confirm g is a primitive root by ensuring $g^i \bmod p \neq 1$ for i from 1 to $p-2$ and $g^{(p-1)} \bmod p = 1$.
- Ensure a , k , and m are within valid ranges and s has a modular inverse.

5. Output:

- Display the public key (p, g, h) , private key a , random k , ciphertext $(c1, c2)$, and decrypted message.
- Verify decryption success if the decrypted message matches the original.

6. Key Considerations:

- The algorithm's security relies on the discrete logarithm problem, with a time complexity of $O(\log p * \log a)$ per modular exponentiation.
- The random k ensures different ciphertexts for the same message, enhancing security.

Source Code:

```
#include <stdio.h>

long long powerMod(long long base, long long exp, long long mod)
{
    long long result = 1;
    while (exp > 0)
    {
        if (exp % 2 == 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return result;
}

long long modInverse(long long a, long long mod)
{
    for (long long i = 1; i < mod; i++)
    {
        if ((a * i) % mod == 1)
            return i;
    }
    return 1;
}

int main()
{
    long long p, g, x, k, m;
    long long y, c1, c2, s, decrypted;
    printf("=====\n");
    printf("|| ElGamal Cryptographic Sys ||\n");
    printf("|| Compiled by :- Jonash Chataut ||\n");
    printf("=====\n\n");
    printf("Enter prime number (p): ");
    scanf("%lld", &p);
    printf("Enter primitive root (g): ");
    scanf("%lld", &g);
    printf("Enter private key (x): ");
    scanf("%lld", &x);
    y = powerMod(g, x, p);
    printf("\nPublic Key (p, g, y) = (%lld, %lld, %lld)\n", p, g, y);
    printf("Enter message (m): ");
    scanf("%lld", &m);
    printf("Enter random key (k): ");
    scanf("%lld", &k);
    c1 = powerMod(g, k, p);
    c2 = (m * powerMod(y, k, p)) % p;
    s = powerMod(c1, x, p);
    decrypted = (c2 * modInverse(s, p)) % p;
    printf("\nCiphertext c1 = %lld\n", c1);
    printf("Ciphertext c2 = %lld\n", c2);
    printf("Decrypted Message = %lld\n", decrypted);
}
```



```
    return 0;  
}
```

Output:

```
C:\csit\5_fifth sem\Jonash Cry  ×  +  ▾  
=====   
||   ElGamal Cryptographic Sys   ||   
|| Compiled by :- Jonash Chataut ||   
=====   
  
Enter prime number (p): 23  
Enter primitive root (g): 5  
Enter private key (x): 6  
  
Public Key (p, g, y) = (23, 5, 8)  
Enter message (m): 10  
Enter random key (k): 3  
  
Ciphertext c1 = 10  
Ciphertext c2 = 14  
Decrypted Message = 10
```